LAB 2:

Experimental Comparison

การทดลองเชิงเปรียบเทียบ

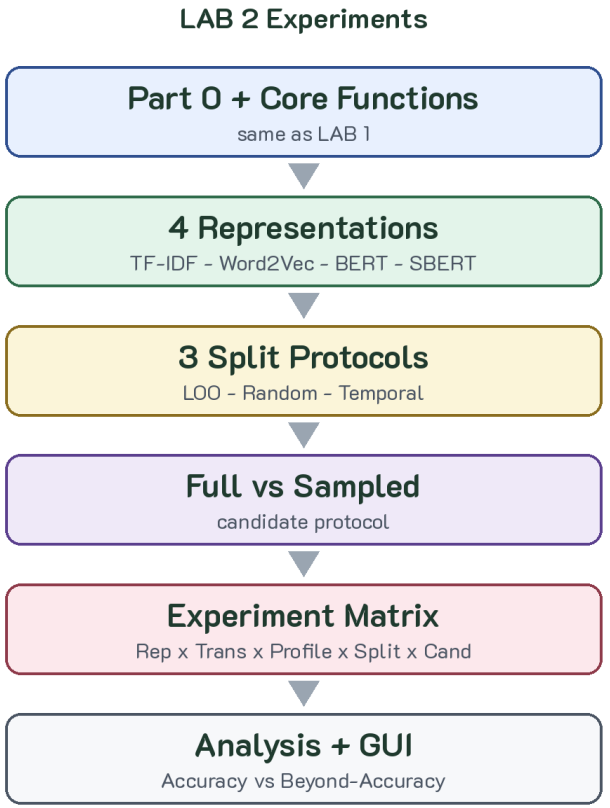
204729

Recommender Systems

ผู้ช่วยศาสตราจารย์ ดร.ดุษฎี ประเสริฐธิติพงษ์

Experimental Comparison

Representations - Protocols



อธิบายแนวคิดของ Source Code ทีละ Part

experiment matrix 5 มิติ บน Core Functions เดิม

COMPUTER SCIENCE

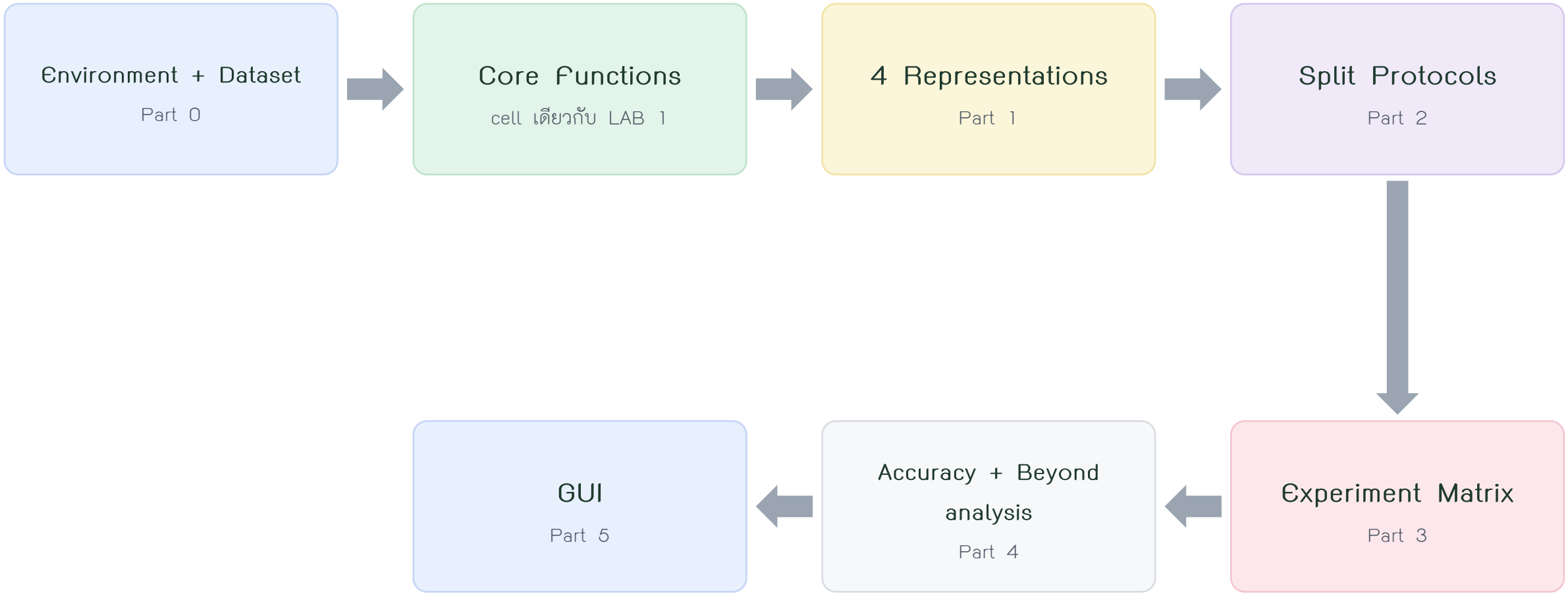
LAB 2 ขยายจาก LAB 1 อย่างไร

ภาพรวม

- **Part 0 และ Core Functions เหมือน LAB 1 ทุกบรรทัด** (transform, 3 profiles, rank_user, 13 metric, loo_folds, evaluate_config) — เล่มนี้ไม่เพิ่มฟังก์ชันหลักใหม่ แต่ **เพิ่มเงื่อนไข** ที่ป้อนเข้าไป
- **มิติใหม่ 1 — Representation:** TF-IDF, Word2Vec (ฝึกจาก 12 เรื่อง), BERT mean pooling, Sentence-BERT ⟶ dict `RAW_REPRESENTATIONS`
- **มิติใหม่ 2 — Split protocol:** LOO (เดิม), Random Holdout (สุ่ม positive 1 เรื่อง), Temporal Holdout (เมื่อมี timestamp) ⟶ dict `SPLITS`
- **มิติใหม่ 3 — Candidate protocol:** Full Catalog vs Sampled Negatives (มีอยู่แล้วใน `evaluate_config` แต่ LAB 1 ใช้เฉพาะ Full)
- รวมกับ Transformation (2) และ Profile (3) ⟶ **experiment matrix 5 มิติ** = 3 × 2 × 3 × 2 × 2 = 72 configuration (ไม่รัน BERT ใน grid เพื่อประหยัดเวลา)
- GUI ให้เลือก Representation / Transformation / Profile / α / β / K แล้วดูผลทันที

หลักการ: ควบคุมเงื่อนไขให้ชัด แล้วอ่านผลเป็น experimental condition ไม่ใช่ model name

โครงสร้าง Notebook LAB 2



Part 1–2 คือตัวแปรต้นใหม่ ; Part 3 วน grid ผ่าน evaluate_config เดิม

แผนที่ของ Source Code LAB 2

Part	หน้าที่	ฟังก์ชัน / ตัวแปรสำคัญ	ต่างจาก LAB 1
0	Environment + Dataset	`items`, `ratings`, `ITEM_INDEX`, `POPULARITY`	เหมือนเดิม
Core	transform / profiles / rank / metrics / folds / evaluate_config	`evaluate_config(X, prof, k, folds, candidate_mode, n_neg, seed)`	เหมือนเดิม (ดูสไลด์ LAB 1)
1	สร้าง 4 representation	`X_tfidf`, `X_w2v`, `X_sbert`, `X_bert`, `RAW_REPRESENTATIONS`	ใหม่
2	Split protocols	`random_holdout_folds`, `temporal_holdout_folds`, `SPLITS`	ใหม่ (LOO มาจาก `loo_folds`)
3	Experiment matrix	`REP_TO_RUN`, `TRANS_TO_RUN`, ..., `experiment_results`	ใหม่: loop 5 ชั้น
4	วิเคราะห์ Accuracy + Beyond	`cols`, sort by Split, nDCG@3	ใหม่
5	GUI	`gui_model`, `profile_preview`	เพิ่ม dropdown Representation ; แสดง profile 20 มิติแรกแทนคำ

1

Core Functions ที่ใช้ซ้ำ

cell เดียวกับ LAB 1 — ทบทวนเฉพาะ interface ที่ LAB 2 เรียกใช้

- `transform_vectors(X, mode)` ⟶ `Raw+L2` / `Center+L2` / `Standard+L2`
- `PROFILE_METHODS` = Positive-only / Rating-centered / Rocchio ผ่าน `get_profile`
- `evaluate_config(X, profile_method, k, folds, candidate_mode, n_neg, seed)` ⟶ `(summary, detail)`

ทบทวน interface ของ Core Functions

```
LIKED_THRESHOLD = 4.0; DISLIKED_THRESHOLD = 2.0

def transform_vectors(X, mode="Raw+L2"): ...       # Raw+L2 | Center+L2
        | Standard+L2
def profile_positive_only(user_id, X, ratings_df): ...          #
        mean(liked)
def profile_rating_centered(user_id, X, ratings_df): ...        # w =
        rating - user mean
def profile_rocchio(user_id, X, ratings_df, alpha=1.0, beta=1.0): ...
        # a*liked - b*disliked
PROFILE_METHODS = {"Positive-only": ..., "Rating-centered": ...,
        "Rocchio": ...}
def get_profile(user_id, X, ratings_df, method, alpha=1.0, beta=1.0):
        ...
def rank_user(user_id, X, ratings_df, method, k=None, candidates=None,
        alpha=1.0, beta=1.0):
    ...; return ranked, score, p

# 8 accuracy/ranking + 5 beyond-accuracy metrics
precision_at_k, recall_at_k, f1_at_k, hr_at_k, reciprocal_rank,
        ap_at_k, ndcg_at_k
coverage, gini_coefficient, ild, novelty, calibration_genre

def loo_folds(ratings_df, positive_threshold=4.0): ...  # [(uid, train,
        heldout)]
def evaluate_config(X, profile_method, k=3, alpha=1.0, beta=1.0,
        folds=None,
                    candidate_mode="Full Catalog", n_neg=20, seed=42):
    ...; return pd.Series(summary), detail
```

อธิบายโค้ด

จุดที่ LAB 2 ใช้ประโยชน์

- `evaluate_config` รับ `folds` จากภายนอก ⟶ ส่ง LOO / Random / Temporal folds เข้าไปได้โดยไม่แก้ engine
- รับ `candidate_mode` + `n_neg` + `seed` ⟶ Full Catalog vs Sampled Negatives ทำซ้ำได้
- รับ `X` ใด ๆ ที่เรียงตาม `items` ⟶ ใส่ Word2Vec/BERT/SBERT แทน TF-IDF ได้ทันที (ILD จะวัดในspace นั้น)
- รับ `profile_method` ชื่อจาก `PROFILE_METHODS` ⟶ วนได้ทั้งสาม
- ผลลัพธ์เป็น `pd.Series` ⟶ `**s.to_dict()` ต่อเป็นแถวของ DataFrame ได้ทันที

2

4 Item Representations

Part 1: TF-IDF • Word2Vec • BERT Mean Pooling • Sentence-BERT

- ทุกวิธีคืนเมทริกซ์ 12 แถวเรียงตาม `items` (ยังไม่ normalize — Part 3 จะ transform)
- เก็บใน `RAW_REPRESENTATIONS`

Part 1: TF-IDF และ Word2Vec

```python
# TF-IDF
tfidf = TfidfVectorizer(lowercase=True, ngram_range=(1, 2), min_df=1)
X_tfidf = tfidf.fit_transform(items["text_content"]).toarray()

# Word2Vec
from gensim.models import Word2Vec
tokens = [str(x).lower().split() for x in items["text_content"]]
w2v = Word2Vec(tokens, vector_size=100, window=5, min_count=1,
        workers=2, epochs=100, seed=42)
def doc_w2v(tok):
    vv = [w2v.wv[w] for w in tok if w in w2v.wv]
    return np.mean(vv, axis=0) if vv else np.zeros(w2v.vector_size)
X_w2v = np.vstack([doc_w2v(t) for t in tokens])
```

อธิบายโค้ด

sparse lexical vs static dense

- TF-IDF: มิติ = คำ/คู่คำ, ค่าไม่ติดลบ, sparse, ตีความได้ (LAB 1)
- Word2Vec: `tokens` ตัดคำด้วย `split()` ธรรมดา (ไม่ตัดเครื่องหมายวรรคตอน) ; CBOW ค่าเริ่มต้น (`sg` ไม่ระบุ), 100 มิติ, `epochs=100`, `seed=42`
- `doc_w2v` = ค่าเฉลี่ยเวกเตอร์คำในเรื่อง ⟶ เวกเตอร์เอกสาร ; คำที่ไม่มีใน vocab ถูกข้าม (แต่ `min_count=1` ทำให้มีทุกคำ)
- **ข้อจำกัดที่ notebook เตือน:** corpus 12 เรื่องเล็กเกินไป — Word2Vec ที่ได้ใช้เพื่อ **สาธิตขั้นตอน** ไม่ใช่คุณภาพ ; ผลใน grid มักแพ้ TF-IDF
- `workers=2` ทำให้ผลอาจไม่ deterministic 100% แม้ตั้ง seed (gensim ต้อง workers=1 จึงจะทำซ้ำได้เป๊ะ)

Part 1: Sentence-BERT และ BERT mean pooling (1/2)

```python
# Sentence-BERT
from sentence_transformers import SentenceTransformer
sbert = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
X_sbert = sbert.encode(items["text_content"].tolist(),
        show_progress_bar=True)

# BERT mean pooling
import torch
from transformers import AutoTokenizer, AutoModel
bert_name = "bert-base-uncased"
bert_tok = AutoTokenizer.from_pretrained(bert_name)
bert = AutoModel.from_pretrained(bert_name); bert.eval()
def bert_embed(texts, batch_size=8):
    rows = []
    for i in range(0, len(texts), batch_size):
        b = texts[i:i + batch_size]
```

อธิบายโค้ด

contextual embedding สองแบบ

- SBERT (`all-MiniLM-L6-v2`, 384 มิติ): fine-tune ให้ cosine ระหว่างประโยคมีความหมาย ⟶ ใช้กับ recommender แบบ cosine ได้ตรง ๆ ; ที่นี่ **ไม่ได้** ตั้ง `normalize_embeddings` เพราะ Part 3 จะ L2 ผ่าน `transform_vectors`
- BERT (`bert-base-uncased`, 768 มิติ): ได้เวกเตอร์ต่อ token ⟶ **mean pooling แบบมี mask** (ไม่นับ padding) ให้เหลือ 1 เวกเตอร์ต่อเรื่อง ; `eval()` + `no_grad()` = inference เท่านั้น
- BERT ดิบมี anisotropy (เวกเตอร์คล้ายกันหมด) ⟶ cosine กระจายแคบ ; Center+L2 ช่วยได้ — เป็นเหตุผลหนึ่งที่ grid มี Transformation 2 แบบ
- `RAW_REPRESENTATIONS` ทั้ง 4 ยังเป็น raw ⟶ Part 3 ตัดสินใจ transformation
- BERT ไม่ถูกใส่ใน `REP_TO_RUN` โดยค่าเริ่มต้น (ช้า) แต่ GUI เลือกได้

Part 1: Sentence-BERT และ BERT mean pooling (2/2)

```
        inp = bert_tok(b, padding=True, truncation=True,
                max_length=128, return_tensors="pt")
        with torch.no_grad():
            h = bert(**inp).last_hidden_state             # (batch,
                    seq, 768)
        mask = inp["attention_mask"].unsqueeze(-1)
        z = (h * mask).sum(1) / mask.sum(1).clamp(min=1)   # masked
                mean pooling
        rows.append(z.numpy())
    return np.vstack(rows)
X_bert = bert_embed(items["text_content"].tolist())

RAW_REPRESENTATIONS = {"TF-IDF": X_tfidf, "Word2Vec": X_w2v, "BERT":
        X_bert, "SBERT": X_sbert}
for name, X in RAW_REPRESENTATIONS.items(): print(name, X.shape)
```

อธิบายโค้ด

contextual embedding สองแบบ

- SBERT (`all-MiniLM-L6-v2`, 384 มิติ): fine-tune ให้ cosine ระหว่างประโยคมีความหมาย ⟶ ใช้กับ recommender แบบ cosine ได้ตรง ๆ ; ที่นี่ **ไม่ได้** ตั้ง `normalize_embeddings` เพราะ Part 3 จะ L2 ผ่าน `transform_vectors`
- BERT (`bert-base-uncased`, 768 มิติ): ได้เวกเตอร์ต่อ token ⟶ **mean pooling แบบมี mask** (ไม่นับ padding) ให้เหลือ 1 เวกเตอร์ต่อเรื่อง ; `eval()` + `no_grad()` = inference เท่านั้น
- BERT ดิบมี anisotropy (เวกเตอร์คล้ายกันหมด) ⟶ cosine กระจายแคบ ; Center+L2 ช่วยได้ — เป็นเหตุผลหนึ่งที่ grid มี Transformation 2 แบบ
- `RAW_REPRESENTATIONS` ทั้ง 4 ยังเป็น raw ⟶ Part 3 ตัดสินใจ transformation
- BERT ไม่ถูกใส่ใน `REP_TO_RUN` โดยค่าเริ่มต้น (ช้า) แต่ GUI เลือกได้

4 Representation ในมุมของการทดลอง

Representation	มิติ	ลักษณะ	คาดหวังใน grid	ILD วัดใน space นี้
TF-IDF	|vocab| (หลายร้อย)	sparse, lexical, ไม่ติดลบ	baseline แข็ง — จับคำซ้ำได้ดีกับ synopsis สั้น	cosine กระจายกว้าง ⟶ ILD สูงเทียม
Word2Vec	100	dense, static, ฝึกจาก 12 เรื่อง	ต่ำสุด — ข้อมูลไม่พอ	ไม่แน่นอน
BERT	768	contextual, mean pooled	cosine แคบ ; ดีขึ้นเมื่อ Center+L2	ILD ต่ำ (anisotropy)
SBERT	384	sentence embedding	จับ semantic ; Coverage มักสูงกว่า TF-IDF	ILD กลาง

ห้ามเทียบค่า ILD ข้าม representation ตรง ๆ — space ต่างกัน

3

Offline Split Protocols

Part 2: LOO • Random Holdout • Temporal Holdout และแนวคิด Candidate Protocol

- ทุก protocol คืน `[(uid, train, heldout)]` รูปแบบเดียวกับ `loo_folds`
- Temporal ทำงานเฉพาะเมื่อมีคอลัมน์ `timestamp` — ไม่สร้างปลอม

Part 2: random / temporal holdout folds และ SPLITS

```python
def random_holdout_folds(ratings_df, positive_threshold=4.0, seed=42):
    rng = np.random.default_rng(seed); folds = []
    for uid, g in ratings_df.groupby("user_id"):
        pos = g[g["rating"] >= positive_threshold]
        if pos.empty: continue
        hold = pos.iloc[int(rng.integers(0, len(pos)))]          # สุ่ม
                positive 1 เรื่อง
        folds.append((str(uid), ratings_df.drop(index=hold.name),
                str(hold["item_id"])))
    return folds

def temporal_holdout_folds(ratings_df, positive_threshold=4.0):
    if "timestamp" not in ratings_df.columns: return None
    folds = []
    for uid, g in ratings_df.groupby("user_id"):
        pos = g[g["rating"] >=
                positive_threshold].sort_values("timestamp")
        if pos.empty: continue
        hold = pos.iloc[-1]                                      #
                positive ล่าสุด
        folds.append((str(uid), ratings_df.drop(index=hold.name),
                str(hold["item_id"])))
    return folds

SPLITS = {"LOO": loo_folds(ratings), "Random Holdout":
        random_holdout_folds(ratings)}
temp = temporal_holdout_folds(ratings)
if temp is not None: SPLITS["Temporal Holdout"] = temp
print({k: len(v) for k, v in SPLITS.items()})
```

อธิบายโค้ด

สามวิธีเลือก held-out

- LOO (`loo_folds` ใน Core): positive ที่ rating **สูงสุด** ⟶ ทดสอบว่าระบบหาหนังที่ผู้ใช้ชอบที่สุดเจอไหม
- Random Holdout: positive **สุ่ม** 1 เรื่อง (seed คงที่) ⟶ ลด bias จากการเลือกเรื่อง rating สูงสุด ; ผู้ใช้ชุดเดียวกับ LOO แต่ held-out อาจต่าง
- Temporal: positive **ล่าสุด** ตาม `timestamp` ⟶ สมจริงที่สุด แต่ MovieMini-12 ไม่มี timestamp ⟶ คืน `None` และไม่ถูกใส่ใน `SPLITS`
- ทั้งสามคืน tuple รูปแบบเดียวกัน ⟶ `evaluate_config(folds=...)` ใช้ได้ทันที
- print จำนวน fold ต่อ protocol ⟶ ในเวอร์ชันนี้ทุก protocol มี 1 fold ต่อผู้ใช้ที่มี positive จึงเท่ากัน

Candidate Protocol: Full Catalog vs Sampled Negatives

Full Catalog

- candidate = ทุกเรื่องที่ไม่อยู่ใน train (~8–10 เรื่อง) + บังคับให้มี held-out
- สมจริง ยากกว่า ⟶ HR/MRR/nDCG ต่ำกว่า, AUC วัดกับ negative ทั้งหมด
- LAB 1 ใช้เฉพาะโหมดนี้

Sampled Negatives (`n_neg=20`)

- candidate = held-out + negative สุ่ม n_neg เรื่องด้วย rng seed 42
- กับ catalog 12 เรื่อง negative มีไม่ถึง 20 ⟶ **ได้ candidate เกือบเท่า Full** จึงผลต่างเล็ก ; กับ catalog ใหญ่ตัวเลขจะสูงกว่า Full ชัดเจน
- notebook เตือน: ห้ามเทียบ metric ระหว่างสอง protocol โดยไม่ระบุ

protocol ทั้งสองถูกวนใน Part 3 เป็นมิติ Candidates

4

Experiment Matrix และการวิเคราะห์

Part 3 – Part 4: loop 5 ชั้น แล้วอ่าน Accuracy คู่กับ Beyond-Accuracy

- `experiment_results` = 1 แถวต่อ configuration
- Part 4 เลือกคอลัมน์และเรียงตาม Split แล้ว nDCG@3

Part 3: กำหนด grid และวน 5 มิติ

```
REP_TO_RUN = ["TF-IDF", "Word2Vec", "SBERT"]          # ไม่รวม BERT
        (ช้า) – เพิ่มได้
TRANS_TO_RUN = ["Raw+L2", "Center+L2"]
PROFILES_TO_RUN = list(PROFILE_METHODS)
SPLITS_TO_RUN = list(SPLITS)
CANDIDATES_TO_RUN = ["Full Catalog", "Sampled Negatives"]

rows = []
for rep_name in REP_TO_RUN:
    for trans in TRANS_TO_RUN:
        X = transform_vectors(RAW_REPRESENTATIONS[rep_name], trans)
        for prof in PROFILES_TO_RUN:
            for split_name in SPLITS_TO_RUN:
                for cand in CANDIDATES_TO_RUN:
                    s, _ = evaluate_config(X, prof, k=3,
                            folds=SPLITS[split_name],
                                        candidate_mode=cand,
                                                n_neg=20, seed=42)
                    rows.append({"Representation": rep_name,
                            "Transformation": trans,
                                 "Profile": prof, "Split": split_name,
                                        "Candidates": cand,
                                 **s.to_dict()})
experiment_results = pd.DataFrame(rows)
display(experiment_results.round(4))
```

อธิบายโค้ด

หนึ่งแถว = หนึ่ง experimental condition

- ลำดับ loop: Representation ⟶ Transformation ⟶ Profile ⟶ Split ⟶ Candidates ; `transform_vectors` ถูกเรียกครั้งเดียวต่อคู่ (rep, trans) แล้วใช้ซ้ำในชั้นใน
- 3 × 2 × 3 × 2 × 2 = **72 แถว** (ถ้ามี Temporal จะเป็น 108) ; แต่ละแถวมี 5 คอลัมน์ระบุเงื่อนไข + 13 metric
- `k=3`, `n_neg=20`, `seed=42` คงที่ ⟶ ต่างกันเฉพาะ 5 มิติที่ตั้งใจ
- รายการ `*_TO_RUN` แก้ได้: เพิ่ม `"BERT"` หรือ `"Standard+L2"` แล้วรันใหม่
- long-format table ⟶ กรองด้วย pandas ให้เปลี่ยนทีละมิติก่อนสรุป (เช่น `experiment_results[(Split=="LOO") & (Candidates=="Full Catalog")]`)

Part 4: วิเคราะห์ Accuracy และ Beyond-Accuracy ร่วมกัน

```
cols = ["Representation", "Transformation", "Profile", "Split",
        "Candidates",
        "HR@3", "MRR", "nDCG@3", "Coverage", "Gini", "ILD", "Novelty",
                "Calibration"]
display(experiment_results[[c for c in cols if c in
        experiment_results.columns]]
        .sort_values(["Split", "nDCG@3"], ascending=[True,
                False]).round(4))
```

อธิบายโค้ด

อ่านสองกลุ่มพร้อมกัน

- เลือก 3 metric ด้าน relevance (HR@3, MRR, nDCG@3) และ 5 ด้าน beyond (Coverage, Gini, ILD, Novelty, Calibration) ให้ตารางอ่านได้
- เรียงภายในแต่ละ Split ตาม nDCG@3 ⟶ เห็นว่า configuration ใดจัดอันดับดีที่สุด **ภายใต้ protocol เดียวกัน** (ไม่ปนข้าม Split)
- **สิ่งที่ควรมองหา:** โมเดลที่ nDCG สูงแต่ Coverage ต่ำ/Gini สูง = แม่นแต่แนะนำซ้ำ ; SBERT อาจ Coverage สูงกว่าแต่ MRR ต่ำกว่า TF-IDF
- Calibration สูง = สัดส่วนแนวหนังใกล้ประวัติ (อาจเป็น filter bubble) ; Novelty สูง = แนะนำเรื่องที่คนดูน้อย
- `[c for c in cols if c in ...]` กันกรณีชื่อคอลัมน์ต่างเมื่อเปลี่ยน `k`

คำถามที่ตอบได้จาก experiment_results (กรองทีละมิติ)

Rep × Transformation

- TF-IDF Raw+L2 vs Center+L2: centering ช่วย TF-IDF ไหม?
- SBERT/BERT Center+L2: anisotropy ลดลง ⟶ nDCG ดีขึ้นหรือไม่?
- Word2Vec แพ้ TF-IDF เท่าไร ⟶ ข้อจำกัดของ corpus

Profile

- Rocchio vs Positive-only: negative feedback เปลี่ยนอันดับสำหรับผู้ใช้ที่มี rating ≤ 2 ไหม?
- Rating-centered ให้ Coverage ต่างจากอีกสองแบบหรือไม่?
- α, β ใน grid คงที่ = 1 ⟶ ลองเปลี่ยนผ่าน `evaluate_config(alpha=, beta=)`

Split × Cand.

- LOO vs Random Holdout: held-out ต่างกัน ⟶ ตัวเลขต่างเพราะโจทย์ ไม่ใช่โมเดล
- Sampled vs Full: ต่างน้อยเพราะ catalog เล็ก — สังเกต AUC
- รายงานทุกครั้งต้องระบุ Split + Candidates + K

ตารางเดียวตอบได้หลายคำถาม ถ้าเปลี่ยนทีละตัวแปร

5

GUI: เปลี่ยนเงื่อนไขแล้วดูผลทันที

Part 5: gui_model รับ Representation / Transformation / Profile / α / β / κ

- ใช้ `transform_vectors` + `rank_user` เดิม
- แสดง User History, Profile Preview (20 มิติแรก), Recommendations

Part 5: gui_model และ gr.Interface (1/2)

```
import gradio as gr
def gui_model(user_id, rep_name, trans, profile, alpha, beta, k):
    X = transform_vectors(RAW_REPRESENTATIONS[rep_name], trans)
    ranked, score, p = rank_user(str(user_id), X, ratings, profile,
            k=int(k),
                                 alpha=float(alpha), beta=float(beta))
    hist = ratings[ratings["user_id"].astype(str) ==
            str(user_id)].merge(
        items[["item_id", "title", "genre"]], on="item_id", how="left"
    )[["title", "genre", "rating"]].sort_values("rating",
            ascending=False)
    rec = []
    for rank, iid in enumerate(ranked, 1):
        r = items.iloc[ITEM_INDEX[iid]]
        rec.append({"Rank": rank, "Title": r["title"], "Genre":
                r["genre"],
                    "Score": round(float(score[ITEM_INDEX[iid]]), 4),
                            "Synopsis": r["synopsis"]})
    profile_preview = pd.DataFrame({"dimension": np.arange(min(20,
            len(p))), "value": p[:min(20, len(p))]})
```

อธิบายโค้ด

ต่างจาก GUI ของ LAB 1 สองจุด

- เพิ่ม dropdown Representation ⟶ `transform_vectors(RAW_REPRESENTATIONS[rep], trans)` คำนวณ space ใหม่ทุกครั้งที่กด (เร็วพอกับ 12 เรื่อง) ; Transformation มี Standard+L2 ด้วยแม้ grid ไม่ได้รัน
- Profile Preview แสดงค่า 20 มิติแรกของโปรไฟล์แทน `top_profile_terms` เพราะ Word2Vec/BERT/SBERT ไม่มีชื่อมิติ — ใช้ดูว่าโปรไฟล์เปลี่ยนเมื่อสลับ profile/transformation หรือไม่
- ส่วนที่เหลือเหมือน LAB 1: `rank_user` บน `ratings` เต็ม, ตาราง History และ Recommendations พร้อม Score
- แนวทางสอน: ผู้ใช้เดิม ⟶ สลับ TF-IDF ⟷ SBERT ⟶ ดูว่า Top-K และช่วงของ Score ต่างกันอย่างไร (SBERT/BERT score สูงและใกล้กัน)

Part 5: gui_model และ gr.Interface (2/2)

```
    return hist, profile_preview, pd.DataFrame(rec)

demo = gr.Interface(
    fn=gui_model,
    inputs=[gr.Dropdown(sorted(ratings["user_id"].unique()),
            label="User"),
            gr.Dropdown(list(RAW_REPRESENTATIONS), value="TF-IDF",
                    label="Representation"),
            gr.Dropdown(["Raw+L2", "Center+L2", "Standard+L2"],
                    value="Raw+L2", label="Transformation"),
            gr.Dropdown(list(PROFILE_METHODS), value="Positive-only",
                    label="Profile"),
            gr.Slider(0, 2, 1, 0.1, label="Rocchio alpha"),
            gr.Slider(0, 2, 1, 0.1, label="Rocchio beta"),
            gr.Slider(1, 5, 3, 1, label="Top-K")],
    outputs=[gr.Dataframe(label="User History"),
             gr.Dataframe(label="User Profile Preview"),
             gr.Dataframe(label="Recommendations")],
    title="Chapter 3 – Experimental Content-Based Recommender")
demo.launch(debug=False)
```

อธิบายโค้ด

ต่างจาก GUI ของ LAB 1 สองจุด

- เพิ่ม dropdown Representation ⟶ `transform_vectors(RAW_REPRESENTATIONS[rep], trans)` คำนวณ space ใหม่ทุกครั้งที่กด (เร็วพอกับ 12 เรื่อง) ; Transformation มี Standard+L2 ด้วยแม้ grid ไม่ได้รัน
- Profile Preview แสดงค่า 20 มิติแรกของโปรไฟล์แทน `top_profile_terms` เพราะ Word2Vec/BERT/SBERT ไม่มีชื่อมิติ — ใช้ดูว่าโปรไฟล์เปลี่ยนเมื่อสลับ profile/transformation หรือไม่
- ส่วนที่เหลือเหมือน LAB 1: `rank_user` บน `ratings` เต็ม, ตาราง History และ Recommendations พร้อม Score
- แนวทางสอน: ผู้ใช้เดิม ⟶ สลับ TF-IDF ⟷ SBERT ⟶ ดูว่า Top-K และช่วงของ Score ต่างกันอย่างไร (SBERT/BERT score สูงและใกล้กัน)

สรุปแนวคิดของ Source Code LAB 2

Take-away

- **ไม่เพิ่ม engine เพิ่มเงื่อนไข:** Core Functions เดิม + dict ใหม่สามตัว (`RAW_REPRESENTATIONS`, `SPLITS`, และ candidate mode) ⟶ experiment matrix 5 มิติ
- **Representation 4 แบบต่างกันที่ความหมายของเวกเตอร์:** TF-IDF (คำ), Word2Vec (เฉลี่ยคำ, corpus เล็ก), BERT (mean pooling, anisotropy), SBERT (semantic) ; ทั้งหมดเป็น raw แล้วให้ `transform_vectors` จัดการ
- **Split protocol เปลี่ยนโจทย์ ไม่ใช่โมเดล:** LOO = ชอบที่สุด, Random = สุ่ม, Temporal = ล่าสุด (ต้องมี timestamp จริง)
- **Candidate protocol เปลี่ยนความยาก:** Sampled Negatives ทำให้ตัวเลขดูดีขึ้น ⟶ ระบุเสมอ
- **`evaluate_config` เดียว 72 ครั้ง:** k, n_neg, seed คงที่ ⟶ ผลต่างระหว่างแถวมาจากมิติที่ตั้งใจ
- **อ่าน HR/MRR/nDCG คู่กับ Coverage/Gini/ILD/Novelty/Calibration** และกรองทีละมิติก่อนสรุป ; GUI ใช้ฟังก์ชันเดียวกับการทดลองจึงเชื่อมตัวเลขกับสิ่งที่เห็นได้

LAB 2 = วิธีคิดแบบ controlled experiment สำหรับ content-based recommender